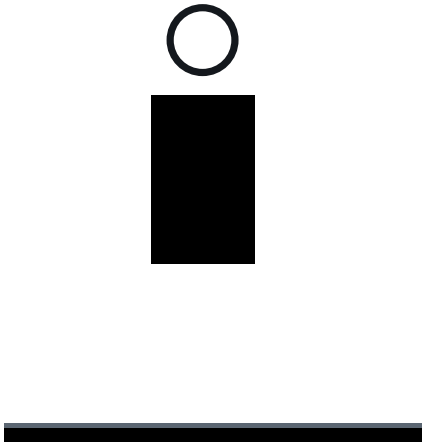
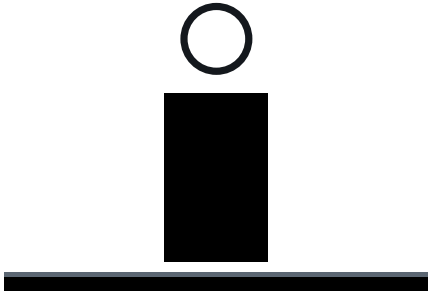
接地判定の方法っていろいろあるんですよ

seika

ジャンプできるのはどっち？



空中



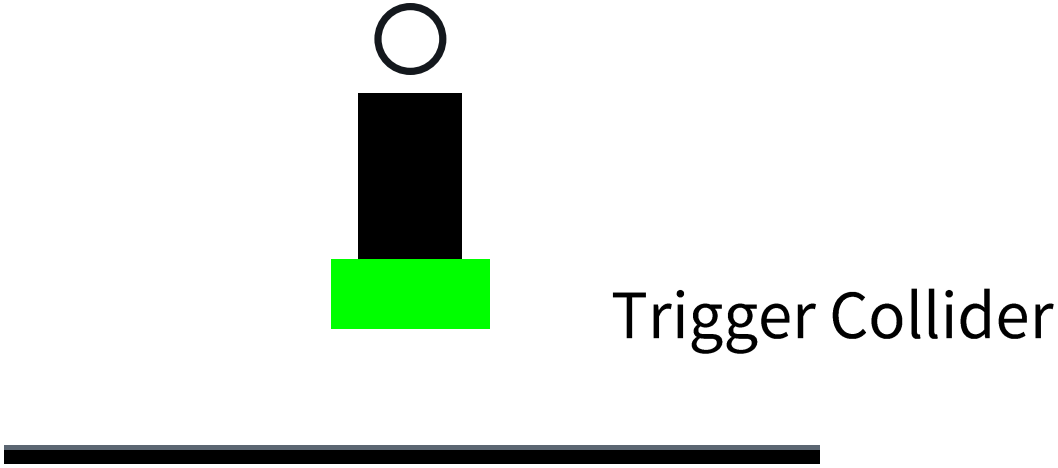
接地

接地判定の4つの方法

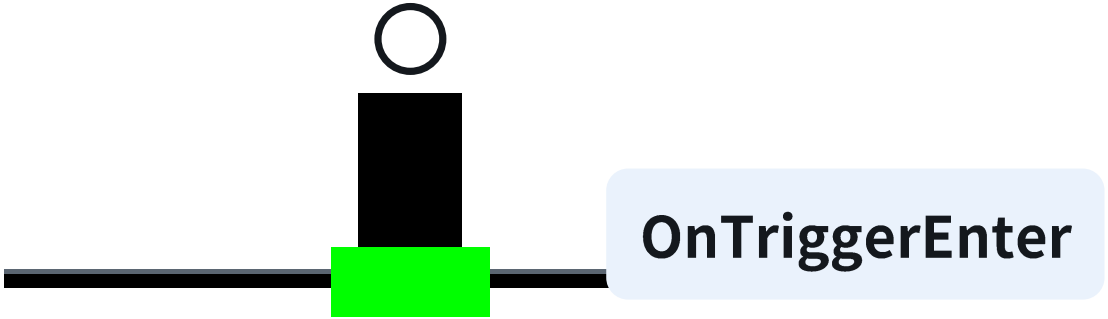
Trigger	Raycast	SphereCast	接触法線
重なり	線	球	面の向き

他にもあるかもね〜

Trigger：足元にセンサーを置く



Trigger：重なったら接地



Trigger：コード

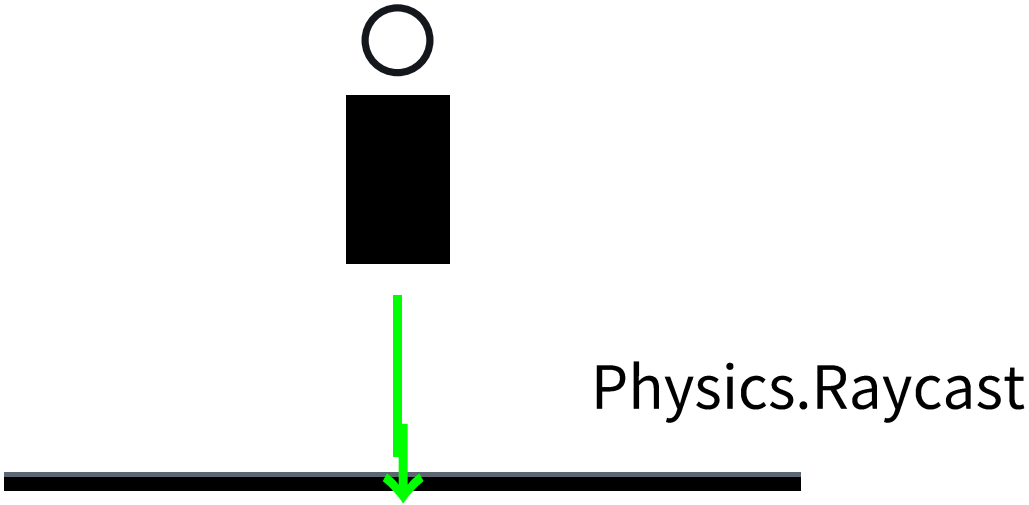
```
private readonly HashSet<Collider> _hits = new();

private void OnTriggerEnter(Collider c) => _hits.Add(c);
private void OnTriggerExit(Collider c) => _hits.Remove(c);

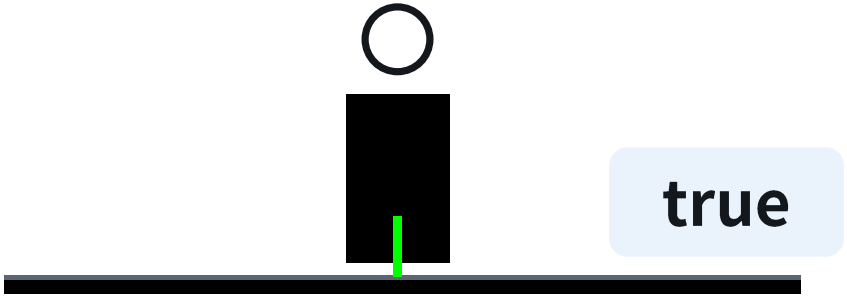
public bool IsGrounded => _hits.Count > 0;
```

欠点：壁との重なりも検出する

Raycast：下向きに線を飛ばす



Raycast：床に当たれば true

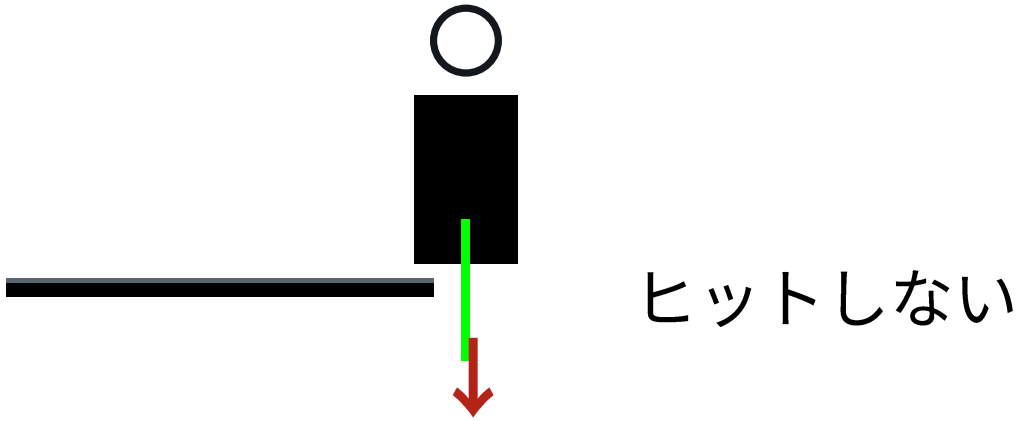


Raycast：コード

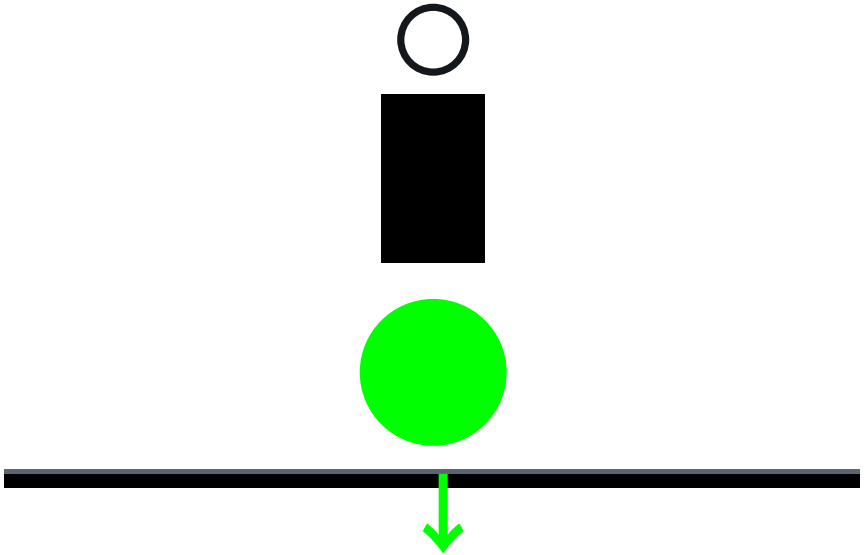
```
Vector3 origin = foot.position + Vector3.up *
0.05f;

bool isGrounded = Physics.Raycast(
    origin, Vector3.down, 0.15f
);
```

Raycast：段差の端に弱い



SphereCast：線ではなく球



SphereCast：端も捉えやすい



true

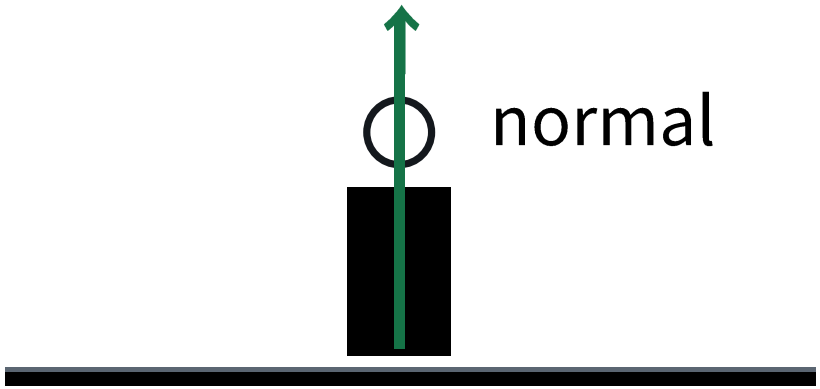
欠点：開始時点で重なっているColliderは検出されない

SphereCast：コード

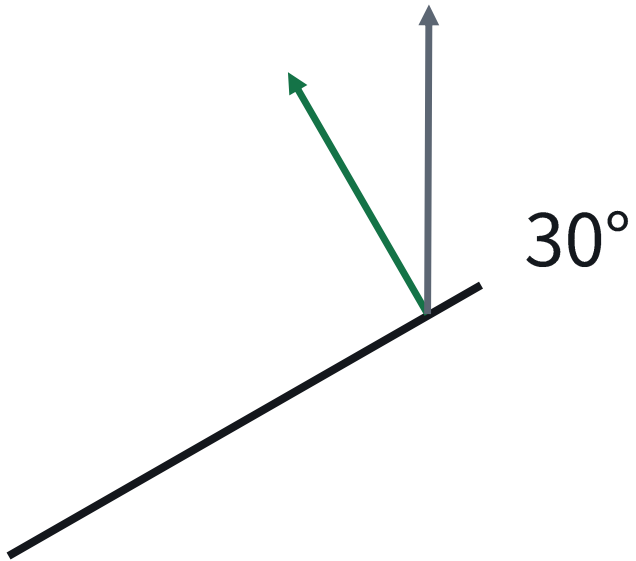
```
Vector3 origin = foot.position + Vector3.up * 0.30f;

bool isGrounded = Physics.SphereCast(
    origin,
    0.25f,
    Vector3.down,
    out _,
    0.15f
);
```

接触法線：面に垂直なベクトル

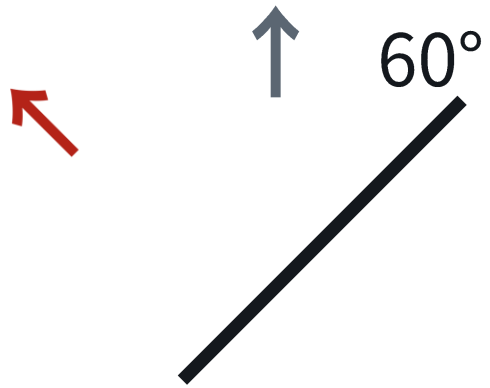


30°の斜面：歩ける



30° ≦ 45°

60°の斜面：歩けない



60° > 45°

接触法線：角度の求め方

$$n \cdot up = \cos\theta$$

$$\theta = \mathrm{acos}(n \cdot up)$$

$$\theta \leq 45°$$

接触法線：接触点を調べる

```
private void OnCollisionStay(Collision c)
{
    bool grounded = false;
    for (int i = 0; i < c.contactCount; i++)
    {
        Vector3 n = c.GetContact(i).normal;
        if (Vector3.Angle(n, Vector3.up) <= 45f)
        {
            grounded = true;
            break;
        }
    }
    if (grounded) _grounds.Add(c.collider);
    else _grounds.Remove(c.collider);
}
```

接触法線：接触先を管理する

```
private readonly HashSet<Collider> _grounds = new();

private void OnCollisionExit(Collision c)
    => _grounds.Remove(c.collider);

public bool IsGrounded => _grounds.Count > 0;
```

まとめ：何を調べたい？

Trigger	重なっているか
Raycast	下に床があるか
SphereCast	幅を持って床を探す
接触法線	歩ける斜面か